

EXP No: 1	Embedded program using Arduino to control the stepper motor
Date: 28.01.2026	

Aim:

To interface and control a stepper motor using Arduino Uno through the L293D Motor Driver IC by writing an embedded program and simulating the circuit using Tinkercad.

Software Required:

1. Tinkercad – Used to design, simulate and to write and compile the embedded C program for Arduino.

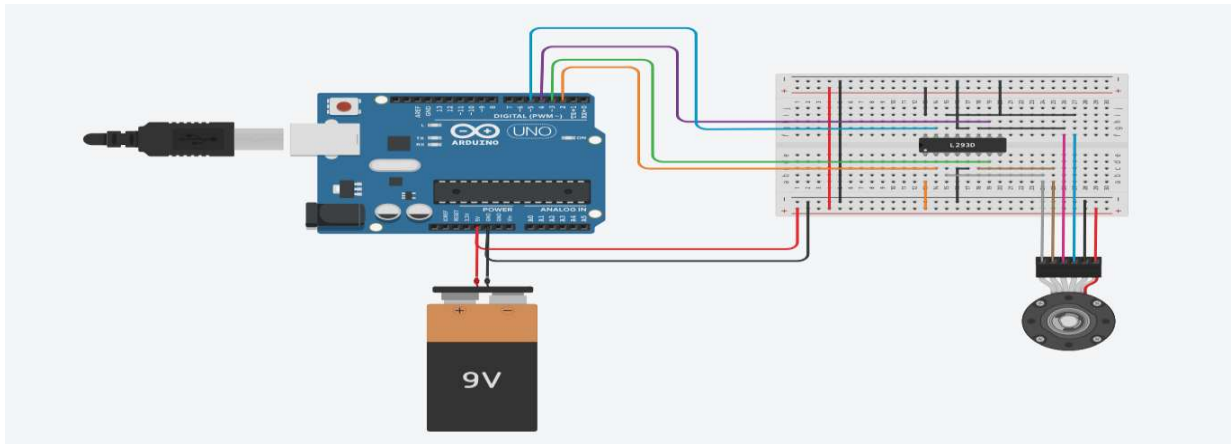
Components Required:

1. Arduino Uno
2. L293D Motor Driver IC
3. Stepper Motor / DC Motor with Encoder
4. Breadboard
5. 9V Battery

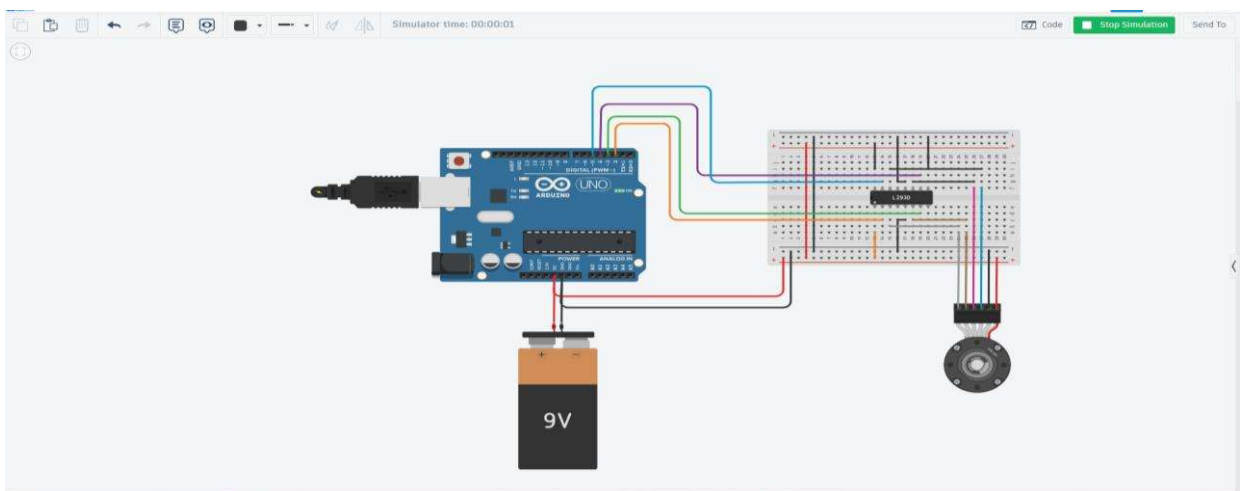
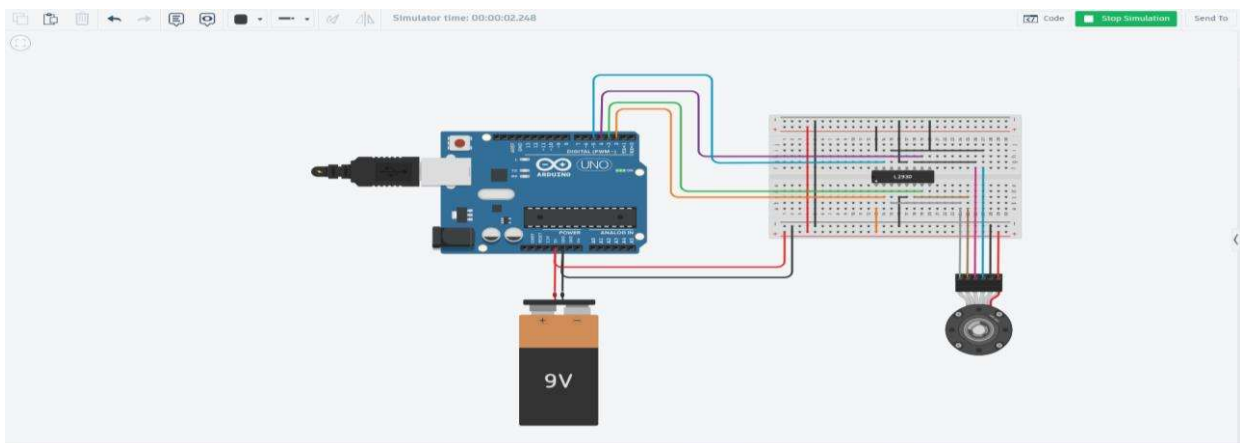
Procedure:

1. Open Tinkercad and create a new circuit simulation.
2. Place the Arduino Uno, L293D motor driver IC, breadboard, stepper motor, and 9V battery in the workspace.
3. Connect the 5V and GND pins of the Arduino to the power rails of the breadboard.
4. Insert the L293D IC on the breadboard and connect its Vcc and GND pins to the power supply rails.
5. Connect the input pins of L293D to the digital pins (D2, D3, D4, D5) of the Arduino Uno.
6. Connect the output pins of L293D to the four terminals of the stepper motor.
7. Provide external power supply to the motor using the 9V battery.
8. Write the Arduino program to energize the motor coils in sequence.
9. Start the simulation in Tinkercad and observe the stepper motor rotation.

Circuit:



Output:



Program:

```
int IN1 = 8;

int IN2 = 9;

int IN3 = 10;

int IN4 = 11;

void setup () {

pinMode(IN1, OUTPUT);

pinMode(IN2, OUTPUT);

pinMode(IN3, OUTPUT);

pinMode(IN4, OUTPUT);

}

void loop() {

digitalWrite(IN1, HIGH);

digitalWrite(IN2, LOW);

digitalWrite(IN3, LOW);

digitalWrite(IN4, LOW);

delay(10);

digitalWrite(IN1, LOW);

digitalWrite(IN2, HIGH);

digitalWrite(IN3, LOW);

digitalWrite(IN4, LOW);

delay(10);

digitalWrite(IN1, LOW);

digitalWrite(IN2, LOW);
```

```
digitalWrite(IN3, HIGH);  
digitalWrite(IN4, LOW);  
delay(10);  
digitalWrite(IN1, LOW);  
digitalWrite(IN2, LOW);  
digitalWrite(IN3, LOW);  
digitalWrite(IN4, HIGH);  
delay(10); }
```

Result:

The stepper motor was successfully controlled using Arduino Uno and L293D motor driver, and the motor rotated according to the programmed sequence in the simulation using Tinkercad.

EXP No: 2	Embedded program using Arduino for timer control operation
Date: 04.02.2026	

Aim:

To develop an embedded program using Arduino Uno to perform timer-based control of LEDs and simulate the operation using Tinkercad.

Software Required:

1. Tinkercad – Used to design, simulate and to write and compile the embedded C program for Arduino.

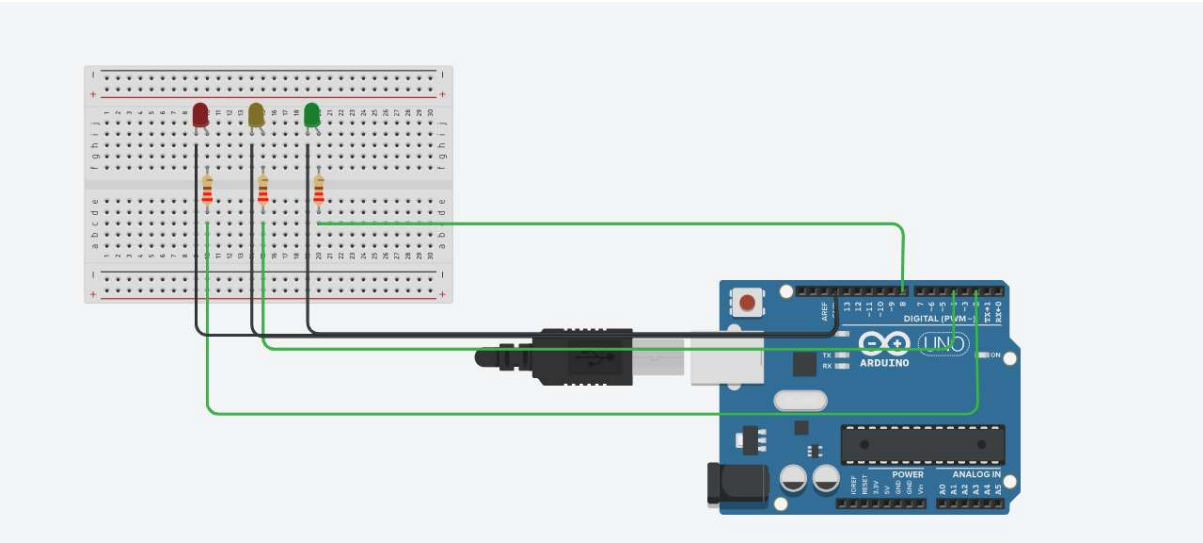
Components Required:

1. Arduino Uno
2. Resistor 1K
3. LEDs

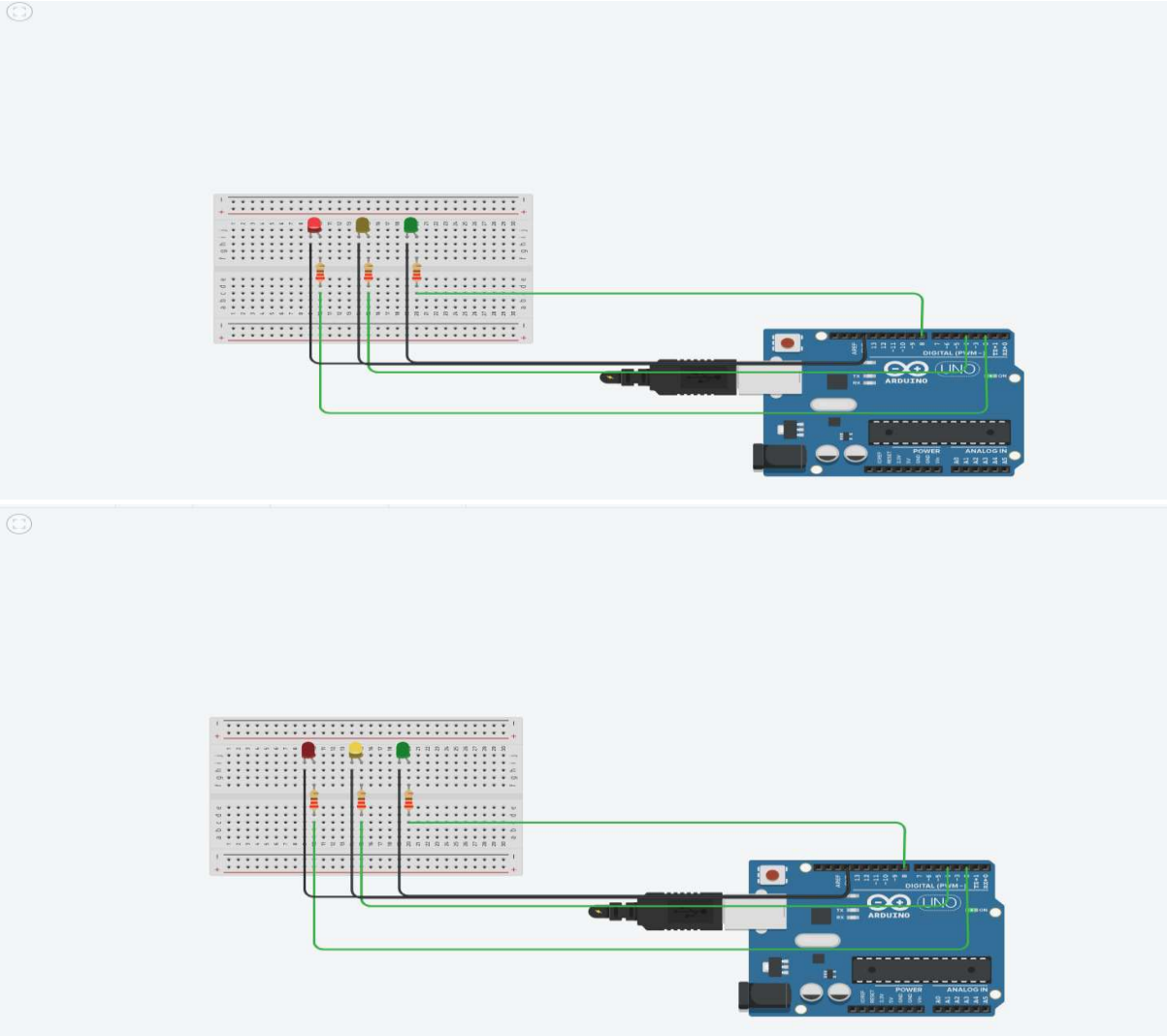
Procedure:

1. Open Tinkercad Circuits and create a new circuit.
2. Place the Arduino Uno, breadboard, three LEDs, and resistors on the workspace.
3. Connect each LED in series with a resistor on the breadboard to limit current.
4. Connect the anode terminals of LEDs to the digital pins of Arduino (D2, D4, and D8).
5. Connect the cathode terminals of LEDs to the GND of the Arduino.
6. Write the Arduino program to switch ON and OFF the LEDs using time delay functions.
7. Upload the program or start the simulation in Tinkercad.
8. Observe the LEDs turning ON and OFF sequentially according to the programmed timer delay.

Circuit:

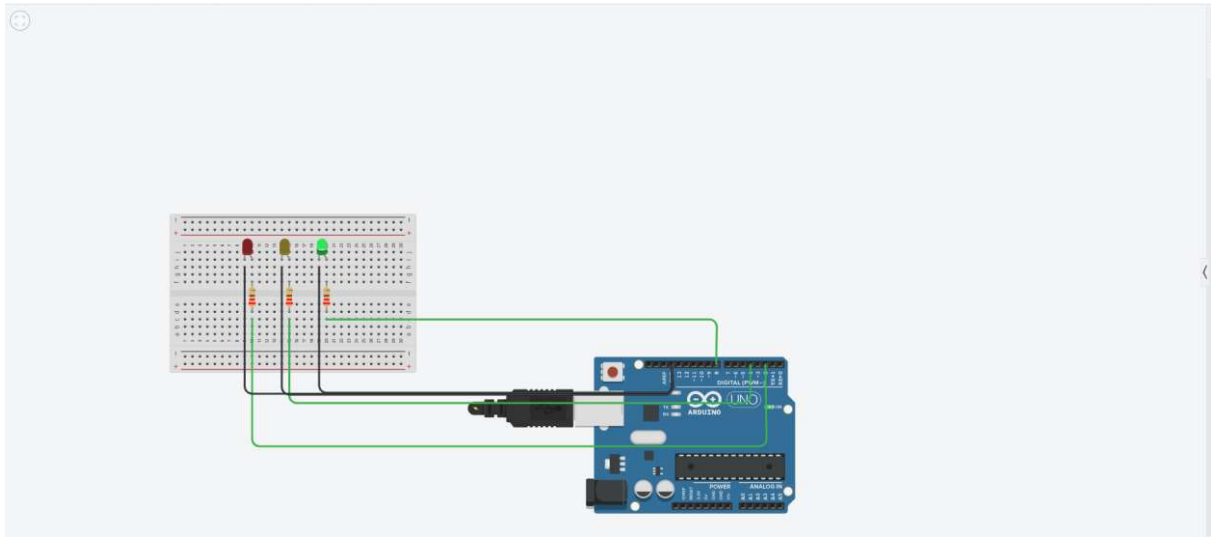


Output



Program:

```
void setup()
{
  pinMode(LED_BUILTIN, OUTPUT);
}
void loop()
{
  digitalWrite(LED_BUILTIN, HIGH);
  delay(2000); // Wait for 1000 millisecond(s)
  digitalWrite(LED_BUILTIN, LOW);
  delay(2000); // Wait for 1000 millisecond(s)
}
```



Result:

The timer control operation was successfully implemented using Arduino Uno, where the LEDs turned ON and OFF sequentially according to the programmed time delay in Tinkercad.

EXP No: 3	Embedded program using Arduino for control the lighting system
Date: 11.02.2026	

Aim:

To design and implement an embedded program using Arduino Uno to control the brightness of an LED using a potentiometer and simulate the circuit using Tinkercad.

Software Required:

1. Tinkercad – Used to design, simulate and to write and compile the embedded C program for Arduino.

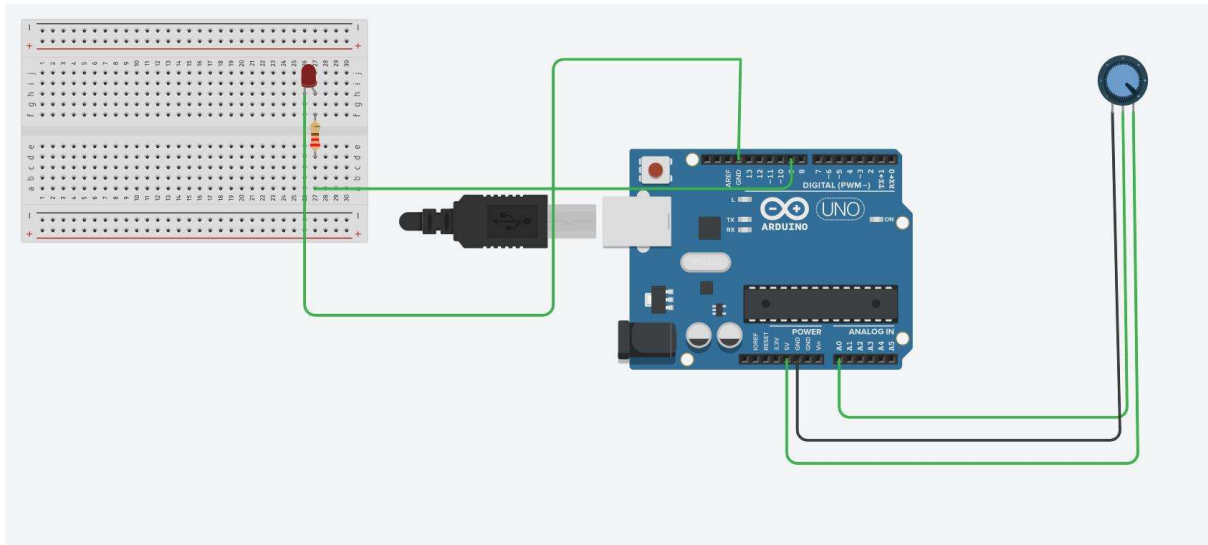
Components Required:

1. Arduino Uno
2. Potentiometer
3. LED
4. Resistor (220Ω / 330Ω)
5. Breadboard
6. Connecting Wires
7. USB Power Supply

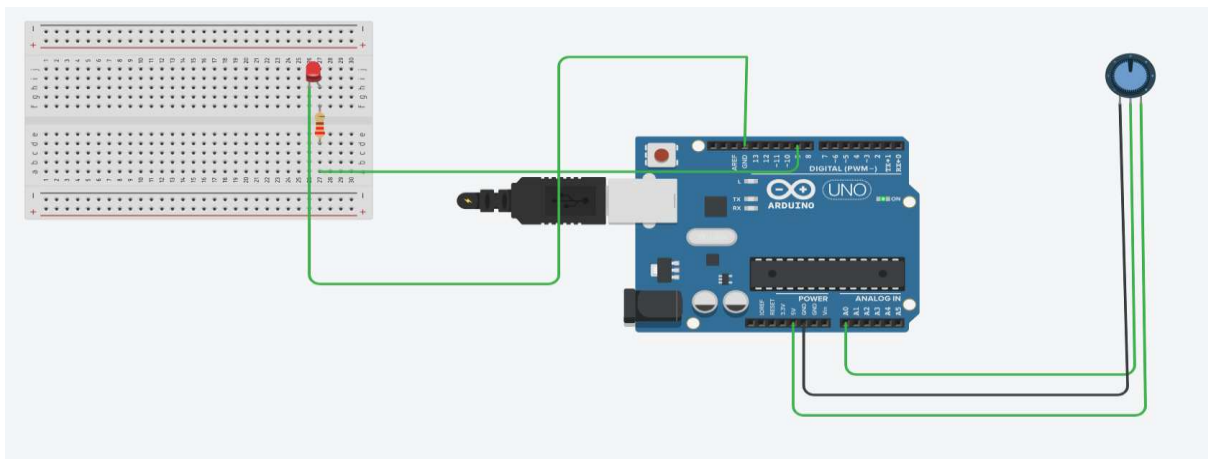
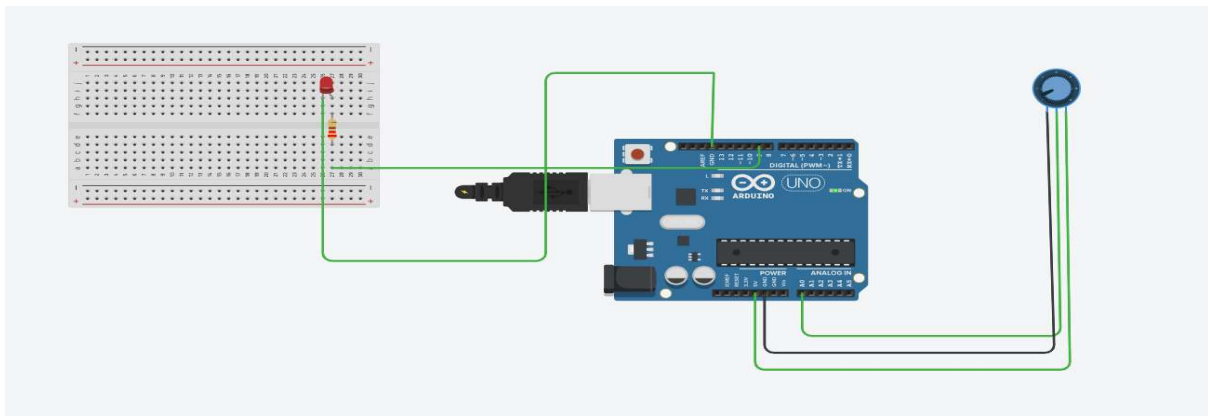
Procedure:

1. Open Tinkercad and create a new circuit.
2. Place the Arduino Uno, breadboard, LED, resistor, and potentiometer in the workspace.
3. Connect the anode of the LED to a digital pin (D9) of the Arduino through a resistor.
4. Connect the cathode of the LED to GND.
5. Connect the potentiometer terminals:
 - i) One terminal to 5V
 - ii) One terminal to GND
6. Middle terminal of the potentiometer to analog pin A0 of Arduino.
7. Write the Arduino program to read the analog value from the potentiometer.
8. Start the simulation in Tinkercad and rotate the potentiometer knob.
9. Observe the change in brightness of the LED based on the potentiometer position.

Circuit:



Output:



Program:

```
#define potPin A0

#define ledPin 9

void setup() {

  pinMode(ledPin, OUTPUT);

}

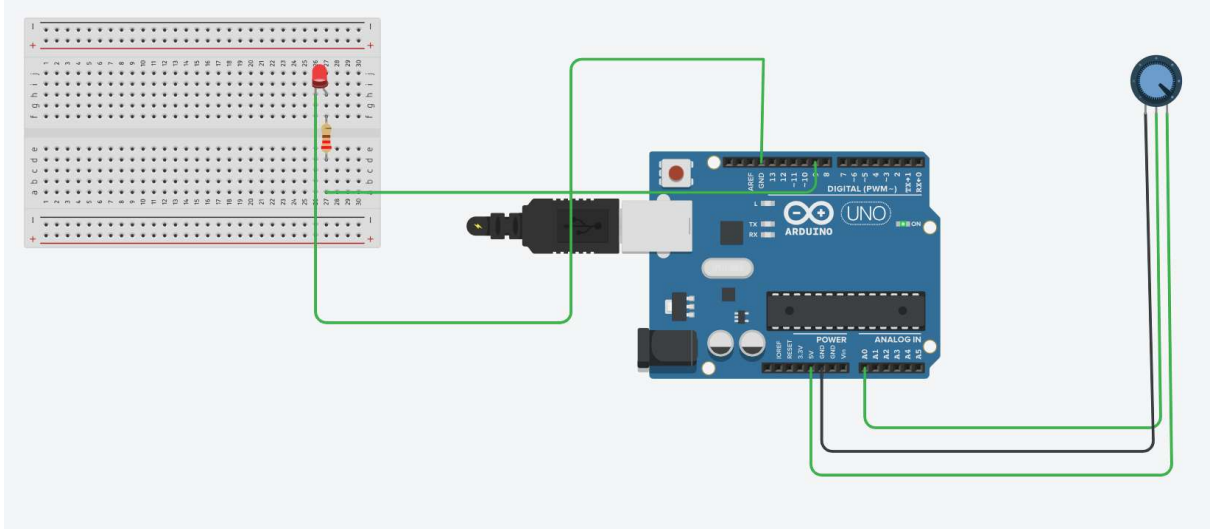
void loop() {

  int potValue = analogRead(potPin);

  int ledBrightness = map(potValue, 0, 1023, 0, 255);

  analogWrite(ledPin, ledBrightness);

}
```



Result:

The lighting system was successfully controlled using Arduino Uno. The brightness of the LED varied according to the position of the potentiometer during simulation in Tinkercad.

EXP No: 4	Embedded program using Arduino for controlling the temperature
Date: 25.02.2026	

Aim:

To measure and monitor temperature using an LM35 Temperature Sensor interfaced with Arduino Uno and display the temperature value on the Serial Monitor using an Arduino program.

Software Required:

1. Tinkercad – Used to design, simulate and to write and compile the embedded C program for Arduino.

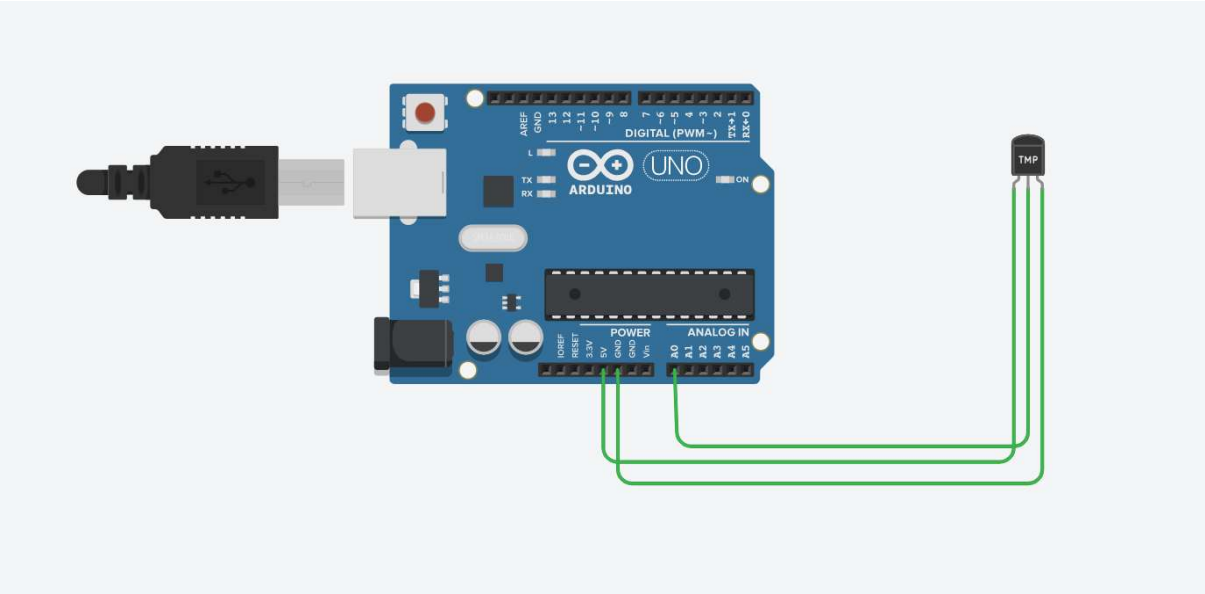
Components Required:

1. Arduino Uno
2. LM35 (temperature sensor)

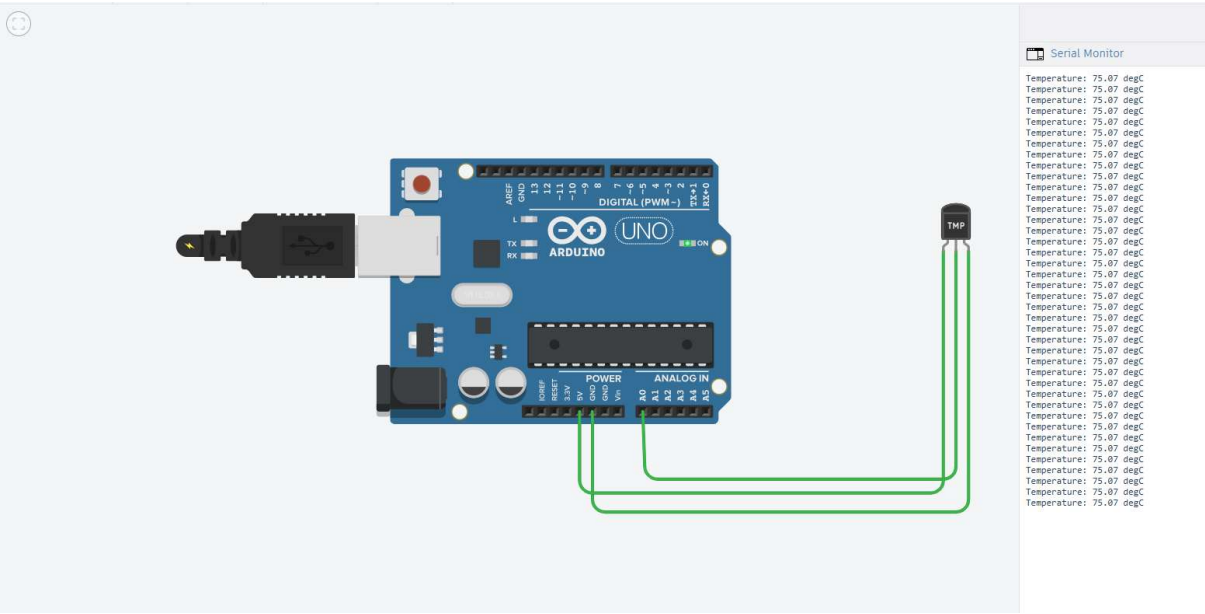
Procedure:

1. Open Tinkercad and create a new circuit simulation.
2. Place the Arduino Uno and LM35 temperature sensor in the workspace.
3. Connect the VCC pin of LM35 to the 5V pin of Arduino.
4. Connect the GND pin of LM35 to the GND pin of Arduino.
5. Connect the output pin of LM35 to the analog input pin A0 of Arduino.
6. Write the Arduino program to read the analog voltage from the LM35 sensor.
7. Convert the analog value into temperature in degrees Celsius using the program.
8. Display the temperature value in the Serial Monitor.
9. Run the simulation and observe the temperature readings displayed on the Serial Monitor of tinkercad.

Circuit:



Output;



Program:

```
int tempPin = A0; // Analog pin connected to temperature sensor

float voltage = 0.0;

float temperatureC = 0.0;

void setup()

{

pinMode(A0, INPUT);

Serial.begin(9600);

}

void loop()

{

int analogValue = analogRead(tempPin);

voltage = (512* 1.5) / 1023.0; // Convert ADC value to voltage

temperatureC = voltage * 100.0; // For LM35 sensor

Serial.print("Temperature: ");

Serial.print(temperatureC, 2);

Serial.println(" degC");

delay(1000);

}
```

Result:

The temperature was successfully measured using the LM35 sensor interfaced with Arduino Uno, and the temperature readings were displayed on the Serial Monitor of Tinkercad.

EXP No: 5	Embedded program using Arduino for display a message in LCD
Date: 11.03.2026	

Aim:

To interface an LCD Display with I2C Module with Arduino Uno and display a message on the LCD using an Arduino program in Tinkercad.

Software Required:

1. Tinkercad – Used to design, simulate and to write and compile the embedded C program for Arduino.

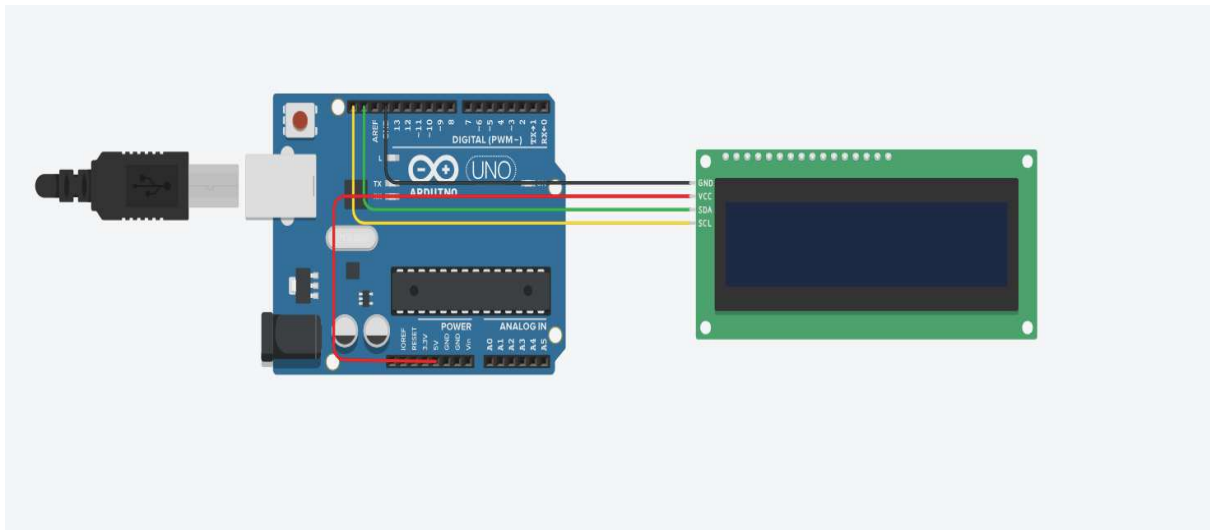
Components Required:

1. Arduino Uno
2. LCD Display with I2C Module

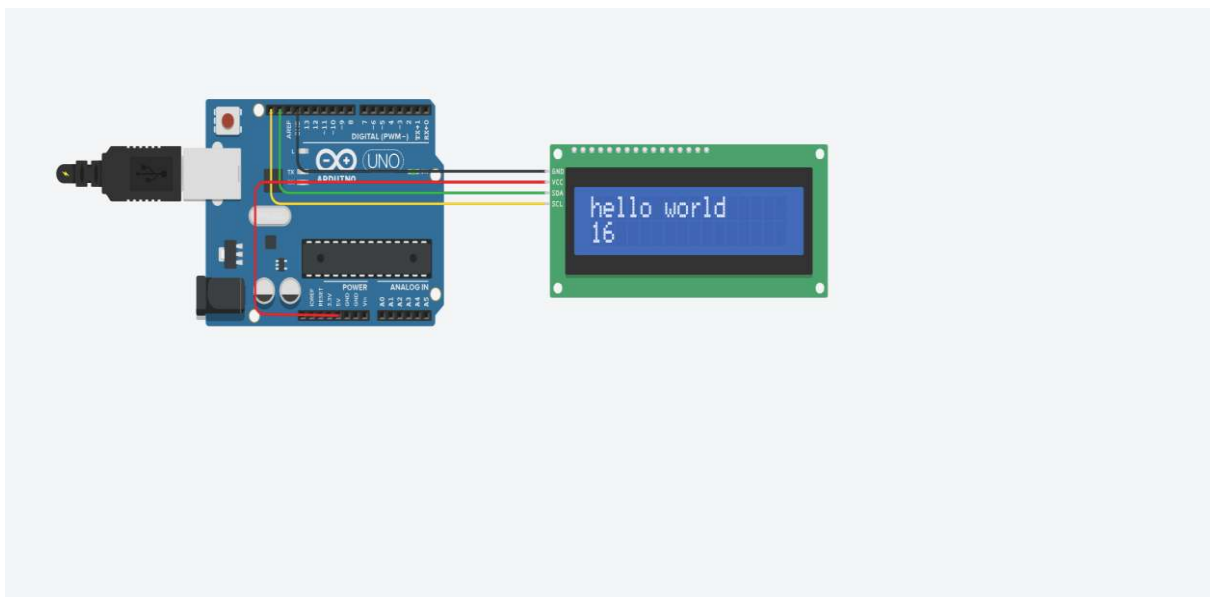
Procedure:

1. Open Tinkercad and create a new circuit simulation.
2. Place the Arduino Uno and LCD with I2C module in the workspace.
3. Connect the VCC pin of the LCD to the 5V pin of Arduino.
4. Connect the GND pin of the LCD to the GND pin of Arduino.
5. Connect the SDA pin of the LCD to the SDA pin of Arduino.
6. Connect the SCL pin of the LCD to the SCL pin of Arduino.
7. Write the Arduino program in Tinkercad to initialize the LCD using the I2C library.
8. Run the simulation and observe the message displayed on the LCD.

Circuit:



Output:



Program:

```
#include <Adafruit_LiquidCrystal.h>
int seconds = 0;
Adafruit_LiquidCrystal lcd_1(0);
void setup()
{
  lcd_1.begin(16, 2);
  lcd_1.print("hello world");
}
void loop()
{
  lcd_1.setCursor(0, 1);
  lcd_1.print(seconds);
  lcd_1.setBacklight(1);
  delay(500); // Wait for 500 millisecond(s)
  lcd_1.setBacklight(0);

  delay(500); // Wait for 500 millisecond(s)
  seconds += 1;
}
```

Result:

The LCD display was successfully interfaced with the Arduino Uno, and the programmed message was displayed on the LCD screen using Tinkercad.